

分块1

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，单点查值。

这是一道用数据结构可以轻松解决的水题，但是如果假设我们不会数据结构，或者考试的时候你写数据结构半天都没有调出来，但是你又觉得裸暴力不太符合你的身份，那么你就可以考虑分块。

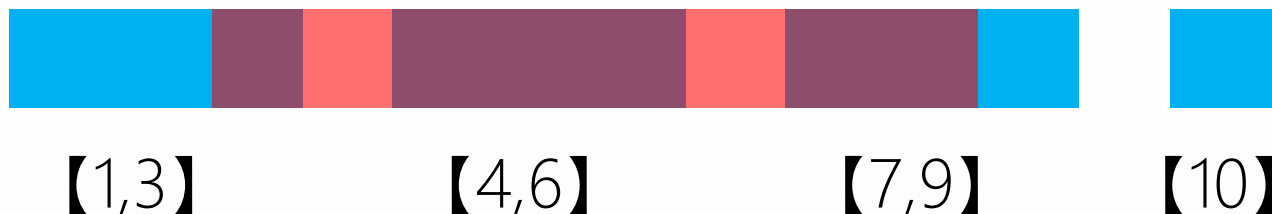
其实分块的思想也就是整体考虑，对于 n 个元素的区间，我们可以分成 m 块，那么区间操作，我们就可以看做几个完整的块的操作和不完整的部分单点操作，对于完整的块的操作，最多操作 m 次，对于不完整的操作，最多操作 $2 \cdot n/m$ 次，很明显，当 $m \cdot m = n$ 的时候，效率最高，大约 $\sqrt{n}$ 。

分块总的时间复杂度为 $n \sqrt{n}$ ，略低于分治

分块1

对于区间修改，我们需要判断当前修改的区间的左右两边的零散区间以及中间的完整区间到底是哪一些。

$n=10$ $k=\sqrt{10}=3;$



比如修改区间 $[2, 8]$ ，实际上就是修改：红色区域。
那么我们就需要先一个一个把 $a[3]$ 修改了，再把 $a[7]$, $a[8]$ 修改了，最后再整体修改第二块 $pos[2]$ 。

分块1

但是分块最大的问题在于，我们怎么判断区间在哪些块中有部分，在哪些块中是一个完整覆盖。实际上很简单，预处理就可以了。

```
for(int i=1;i<=n;i++)  
{  
    pos[i]=(i-1)/k+1;//编号i属于哪一块  
}
```

分块1

修改操作就很简单了，但是要注意不能减重复了，也就是说如果区间左右端点如果在同一个块中，要判断一下。

```
void add(int x,int y,int z)
{
    for(int i=x;i<=min(pos[x]*k,y);i++)
        a[i]+=z;//前面不是一整块的地方一个一个+z
    if(pos[x]!=pos[y])//如果不在一块里面
    {
        for(int i=(pos[y]-1)*k+1;i<=y;i++)
            a[i]+=z;//后面不是一整块的地方+z
    }
    for(int i=pos[x]+1;i<=pos[y]-1;i++)//第i块整体+z
        flag[i]+=z;
}
```

最后第*i*个位置的答案就是 $a[i]+flag[pos[i]]$;

分块2

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，询问区间内小于某个值 x 的元素个数。

输入：

4

1 2 2 3

0 1 3 1

1 1 3 2

1 1 4 1

1 2 3 2

输出：

3

0

2

分块2

该题如果使用以前学习的数据结构貌似还没有可以解决的，区间查询大小，有点类似于主席树，但是很遗憾，我们只学习过静态区间第k大，这种带修改的我们还没有办法处理，至于平衡树，就更没有办法做了。我们这里还是可以考虑分块。

先把n个数分成k块，因为我们要查询区间内小于x的数的个数，所以我们可以把每一个块排序，这样我们就可以用二分查找快速求出每一块中小于x的数的个数了，对于零散的部分，直接一次判断就好。

但是如果块中部分修改之后不满足原来的有序性，所以要保证其有序性，我们每次操作都需要对部分修改的块再次排序，所以效率有点低，大概为 $n \sqrt{n} \log \sqrt{n}$ ，主要就是每次更新之后排序耗费时间

分块2

```
void reset(int x)
{
    v[x].clear();
    for(int i=(x-1)*k+1;i<=min(x*k,n);i++)
        v[x].push_back(a[i]);
    sort(v[x].begin(),v[x].end());
}

void add(int x,int y,int z)
{
    for(int i=x;i<=min(pos[x]*k,y);i++)
        a[i]+=z;
    reset(pos[x]);
    if(pos[x]!=pos[y])
    {
        for(int i=(pos[y]-1)*k+1;i<=y;i++)
            a[i]+=z;
        reset(pos[y]);
    }
    for(int i=pos[x]+1;i<=pos[y]-1;i++)
        flag[i]+=z;
}
```

分块3

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，询问区间内小于某个值 x 的前驱（比其小的最大元素）。

输入：

4

1 2 2 3

0 1 3 1

1 1 4 4

0 1 2 2

1 1 2 4

输出：

3

-1

分块3

又是一道带修改的区间查询的题，如果直接不修改查询前驱，主席树是可以做的，但是一旦带修改，我们目前学习的数据结构是没有办法解决的。

我们还是考虑分块，先分成 k 块，对于每一块，如果要很快的查询前驱，那么很明显的得区间有序，这样我们就可以二分来查找数 x 的前驱，但是又涉及到修改，并且修改之后要需要保证其有序性，我们可以考虑像前面一样排序，但是我们也可以把每一块当做一棵平衡树来实现，即分成 k 棵平衡树，如果手写平衡树很麻烦怎么办，很简单，用set或者multiset。

分块3

对于每一次区间修改，整块修改，还是加标记，部分修改，先把以前的值删除，再修改，再插入。

```
s[pos[x]].erase(a[i]);  
a[i]+=z;  
s[pos[x]].insert(a[i]);
```

分块3

对于每次查询，如果是零散的区域查询，那么很简单，找一个小于x的最大的值。

对于块中查找前驱，即平衡树中查找前驱，直接用set中自带的lower_bound函数，找到 $\geq x$ 的数的之前，前面一个就是前驱，在和以前的前驱比较，选择一个最大的值，但是要注意没有前驱这种情况，即第一个数就是x的后继，那么此时是没有前驱的。

```
int tmp=z-flag[i];
multiset<int>::iterator it=lower_bound(s[i].begin(),s[i].end(),tmp);
if(it==s[i].begin())
continue;
it--;
ans=max(ans,*it+flag[i]);
```

分块4

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，区间求和。

输入：

4

1 2 2 3

0 1 3 1

1 1 4 4

0 1 2 2

1 1 2 4

输出：

1

4



分块4

太简单，解题思路略



分块5

给出一个长为 n 的数列 $a_1 \cdots a_n$ ，以及 n 个操作，操作涉及区间开方，区间求和。

输入：

4

1 2 2 3

0 1 3 1

1 1 4 4

0 1 2 2

1 1 2 4

输出：

6

2

分块5

该题可以用线段树做，但是无论哪种做法都不能暴力修改区间，因为开方和加减不一样，但是一个一个修改肯定会超时，但是值得注意的是最大的数为 2^{32} ，我们一直对他开方， $2^{16}, 2^8, 2^4, 2^2, 2, 1$ 。我们会发现最多6次就会变成1，已经是1的数就不要再开方了，开方答案不会影响的，所以我们标记一下当前区间是否全都是1了，如果是，就不管他，不是，就一个一个修改。最多六次，也就是原有时间复杂度*6。

当然，分块也是一样的思路。

分块5

```
void solve(int x)
{
    if(flag[x])//为1则表示都已经开方到1了
        return ;
    sum[x]=0;
    flag[x]=1;
    for(int i=(x-1)*k+1;i<=x*k;i++)//如果有大于1的就一个一个修改
    {
        a[i]=sqrt(a[i]);
        sum[x]+=a[i];
        if(a[i]>1)
            flag[x]=0;
    }
}
```

分块6

给出一个长为 n 的数列 $a_1 \cdots a_n$ ，以及 n 个操作，操作涉及单点插入，单点查询。数据随机生成

输入：

4

1 2 2 3

0 1 3 1

1 1 4 4

0 1 2 2

1 1 2 4

输出：

2

3

分块6

这道题和前面的题有些不一样，前面的题，不会改变块的结构，只会改变块中元素的值。但是该题涉及块的插入。很容易想到一种分块暴力做法：先分块，记录每一个块的大小，然后查找到插入的块，再去块内查找。为了插入方便，每个块可以开链表，也可以就开不限制长度的数组(vector)。

如果数据随机是可以这样做的，但是如果数据不唯一呢？比如在接近的位置大量插入，那么你的某一个块就会特别大，然后每次查询都查询这个块，很明显，时间复杂度是不可以被接受的。

那么我们就需要重构块了。即如果某一个块的大小超过了一个给定的数值，那么就重新分块。

分块6

```
void rebuild()//重构块
{
    int top=0;
    for(int i=1;i<=m;i++)//枚举每一个块
    {
        for(vector<int>::iterator j=v[i].begin();j!=v[i].end();j++)
            st[++top]=*j;//把这些元素重新提出来开一个数组存储
        v[i].clear();
    }
    int nk=sqrt(top);
    for(int i=1;i<=top;i++)
    {
        pos[i]=(i-1)/nk+1;
        v[pos[i]].push_back(st[i]);
    }
    m=(top-1)/nk+1;
}
```

分块7

给出一个长度为 n 的数列，以及 n 个操作，操作设计区间乘法，区间加法，单点查询

输入：

7

1 2 2 3 9 3 2

0 1 3 1

2 1 3 1

1 1 4 4

0 1 7 2

1 2 6 4

1 1 6 5

2 2 6 4

输出：

3

100

分块7

该题其实原理和线段树2差不多，我们每次操作后仍然可以把乘、加标记处理为两个独立的标记，最后再处理两个独立的乘、加标记。

一：在有乘、加标记的情况下，又来一个乘标记

$$\begin{aligned} & (\text{sum} * a1 + b1) * a2 \\ & = \text{sum} * a1 * a2 + b1 * a1; \end{aligned}$$

也就是相当于把原来的乘标记变为 $a1 * a2$ ，把原来的加标记变为 $b1 * a2$ ；

二：在有乘、加标记的情况下，又来一个加标记

$$\begin{aligned} & (\text{sum} * a1 + b1) + b2 \\ & = \text{sum} * a1 + b1 + b2; \end{aligned}$$

也就是相当于原来的乘标记不变，把原来的加标记变为 $b1 + b2$ ；

分块8

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间询问等于 c 的元素的个数，并将这个区间的所有元素改为 c 。

输入：

4

1 2 2 4

1 3 1

1 4 4

1 2 2

1 4 2

输出：

1

1

0

2

分块8

该题也不是很难，我们可以对每一个块打一个标记，如果该块内元素都为一个值 x 的时候，我们就把该块标记为 x ，否则标记为 -1 。

首先，区间修改的时候，如果这些块有标记，所说明值都是相同的，只要标记不等于 c ，说明就没有相等的，直接修改标记为 c 就可以了。如果这些块没有标记，虽然修改还是一样，但是你没有办法判断中间是否有等于 c 的值，你只能一个一个暴力。对于首尾不完整的地方，先看下所在块是否有标记，如果有，先把原来标记下放，原标记清空，再把涉及到的区间修改为 c